How to Find a Palindrome That Reads

Anonymous ACL submission

Abstract

A palindrome's last letter is fixed by its first, so a left-to-right decoder cannot enforce the constraint by construction: every prefix is extendable, and there is nothing to check until the string ends. We build palindromes from both ends inward and track the *overhang*, the run of letters one side currently owes the other. The overhang is a sufficient statistic for feasibility, which lets standard constrained-decoding machinery attach to a constraint that is useless when posed over prefixes. We then measure what this buys. Progress per placement is not constant: net overhang reduction peaks at 2.03 letters for 3-letter units and falls to 0.37 for units of 10 or more, so long one-sided placements pay down debt only by creating more of it. Composing pre-built palindromic chunks makes length free, and length is therefore not the binding constraint: blind pairwise judgment shows coherence collapses completely at the first seam and does not degrade further as seams are added. A direct count of attested vocabulary shows why material runs out, and a coverage analysis shows that only half of reversed English segments into words at all. Compute is the wrong lever: search throughput falls as $V^{-1.031}$ in vocabulary size. We also show that a punctuation module can make output worse than no punctuation at all, and that the fix is to remove that decision from the search entirely.

1 Introduction

A palindrome reads the same forwards and backwards. That property breaks an assumption every autoregressive decoder relies on: that a string can be extended one token at a time and checked as it grows. In a palindrome, the last letter is fixed the moment the first is chosen, the second-to-last by the second, and so on inward until the two directions meet, so a left-to-right generator cannot enforce this by construction – every prefix, however far along, is still extendable in a way that satisfies it. A model that has written "step on no p" is not one letter closer to a valid palindrome in any way its own prefix can report; it may be one letter away or a thousand, and the prefix does not say which.

This is a clean instance of a problem that recurs across constrained generation: a constraint defined over the finished string, not over the prefix a decoder actually builds, as in rhyme, meter, lexically constrained translation, and reversal tasks. Work on training models to reason about strings backwards (Berglund et al., 2024) and on arithmetic and factorization in reverse order (Kitouni et al., 2024; Golovneva et al., 2024) shows that the direction a model writes in changes what it can represent, not just how fast. Infilling (Donahue et al., 2020; Bavarian et al., 2022), edit-based generation over strings (Edman et al., 2024), and reversal-like benchmarks (Calderaro et al., 2025) all relax or probe the same left-to-right assumption.

The palindrome case already has a textbook answer. Hoey's 1984 algorithm, as described by Norvig (2002), builds from both ends toward the middle instead of left to right. Our contribution is not a new algorithm; it is naming the state that algorithm carries: at any point in the construction, one side of the string owes the other a specific run of letters, which we call the *overhang*. Naming it turns palindrome construction into exactly the shape constrained-decoding machinery was built for: grid beam search, guided and lookahead decoding, lexically constrained decoding, energy-based and MCMC sampling, and work on repairing a model's distribution to respect a constraint (Hokamp and Liu, 2017; Anderson et al., 2017; Post and Vilar, 2018; Lu et al., 2022; Qin et al., 2022; Miao et al., 2019; Zhang et al., 2023; Park et al., 2024; Lew et al., 2023; Loula et al., 2025; Anaya Gonzalez et al., 2025; Gârbacea and Mei, 2025). That machinery conditions on state derived from a prefix; posed over prefixes the palindrome constraint gives it nothing to act

on, but posed over overhangs the constraint is ordinary and finite-state, and the existing toolkit applies without modification.

2 The overhang

Build a palindrome from both ends at once. At any moment the left and right sides under construction are unequal in length or content, and one owes the other a run of letters before the two halves can be declared equal when reversed. That owed run is the overhang, the only thing that needs to be tracked between steps.

Start with left = "step" and right = empty; the overhang is "step" (the right side owes it). Add "on" to the left: left becomes "stepon". Add "pets" to the right: right reversed is "step", which cancels the first four letters of the overhang, leaving "on" outstanding. Add "no" to either side: it cancels "on" exactly, the overhang goes to empty, and the string closes as "step on no pets".

Two partial palindromes can have completely different text so far and still be in the same state, as long as their overhangs match: the same overhang means the same set of valid continuations, independent of how each side got there. That is what makes the overhang a sufficient statistic for feasibility. A candidate lookup table can be built as a trie keyed on the overhang string, and a beam search can be defined directly over the triple (left text, right text, overhang) rather than over the raw string. What changes is that the state driving the search now has a name, a fixed representation, and a lookup structure – the minimum a constrained decoder needs to attach to a constraint at all, and checkable in constant time per step.

3 What one placement buys

Not every placement pays down the same amount of overhang. We sampled 60,000 placements (60,000) from the search's own move distribution and, for each, recorded two quantities: how many letters were *settled* (fixed on both sides by that move) and the *net* change in overhang length.

Figure 1a shows settled letters and net overhang reduction as a function of unit size. Settled letters rise monotonically, from 1.00 for single-letter units to 5.71 for the longest in the sample. Net overhang reduction does not follow: it peaks at 2.03 letters for 3-letter units, then falls to 0.37 for units of 10 or more. Mean net reduction over the sample is 0.90 letters per placement.

The two curves diverging is the finding. A long unit settles more letters than a short one, but it also opens more new constraint on the far side of the overhang than it closes, so the debt it leaves behind grows faster than the debt it pays. Past a certain size, a one-sided placement is worse than useless: the overhang it needs to close later grows faster than the overhang it just closed. A scheme that wants to use long spans has to pay the palindrome constraint *inside* the unit, not after it.

4 Chunks, halves and seams

The alternative is to pay the constraint inside the unit before it ever enters the search. We use the following vocabulary throughout.

Chunk A short palindrome that is already valid on its own, usually 20 letters or more.

LC / RC A chunk split at mirrored positions into a left half and a right half that spells it backwards.

Seam The join where one left chunk follows another before the centre, forcing the corresponding right chunks into reversed order after it.

Nesting works because the mirror relation holds by construction regardless of which chunks are nested: if chunk A splits into LC_A/RC_A and chunk B splits into LC_B/RC_B, then LC_A LC_B ... RC_B RC_A is a palindrome for any choice of A and B, because each half is still the reverse of the other. This makes length free: assembling k chunks costs one seam decision per additional chunk, not a fresh search over letters.

Nesting two chunks our own search found gives a valid 54-letter palindrome. For scale, the longest single readable palindrome we are aware of is 51 letters ("Doc, note: I dissent. A fast never prevents a fatness. I diet on cod."). The comparison is not a claim to have beaten it: our 54-letter text reads worse than either chunk it was built from, which is the point. Length is cheap under composition and says nothing about whether the result reads.

5 Coherence dies at the first seam

Length is free; readability is not. We ran a blind test designed so that material is held fixed and only the number of seams moves. Each item pairs one chunk on its own against a nest built from that same chunk plus $k - 1$ others, for $k \in \{2, 4, 8\}$,

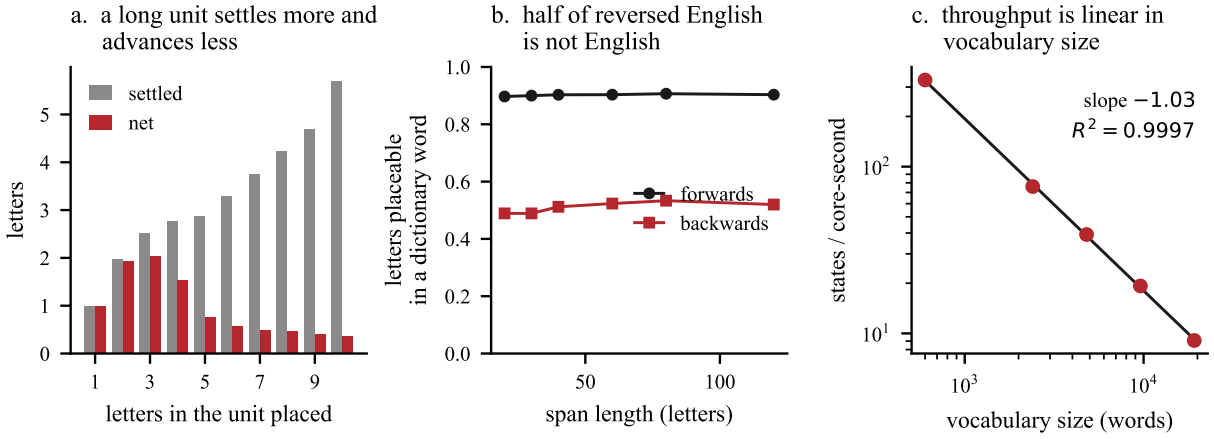

Figure 1: (a) Letters settled and net overhang reduction per placement, by unit size, over 60,000 placements: net reduction peaks at 2.03 for 3-letter units and decays to 0.37 for units of 10 or more. (b) Fraction of a span's letters placeable inside a dictionary word, read forwards versus backwards, against span length: forward coverage stays near 0.90–0.91 while reverse coverage falls to 0.49–0.53. (c) Search throughput in states per core-second against vocabulary size, log-log, over a 32-fold span: the fitted slope is -1.031 ($R^2 = 0.9997$, $n = 5$).

with 7 items per value of k. Two judges scored each pair; the second judge saw every pair with the order flipped, so a judge answering by position alone would agree with itself on 0 of n items, while a judge reading the text would agree on n of n. Both orderings were scored, so a 7-item arm contributes 14 pairwise decisions. Calibration items – real prose against its own word shuffle – were scored under the same protocol to confirm the instrument has power before trusting its verdict on palindromes.

The single chunk was preferred over the nest in all 14 decisions at $k = 2$, all 14 at $k = 4$, and all 14 at $k = 8$. Calibration hit 12 of 12. All results are significant at $p < 0.001$. The two judges agreed on every item, and left-pick rates stayed near half in every arm, so neither position bias nor judge disagreement explains the result.

The result in Figure 2a is a step function, not a decay curve. Going from one chunk to two costs everything a judge is willing to grant; going from two chunks to eight costs nothing further, since the win rate is already 14/14 at $k = 2$ and stays there. A 236-letter eight-chunk nest reads no worse, on this measure, than a 60-letter two-chunk one. If coherence loss scaled with length or with seam count, the win rate should climb or the item-level judgments should split as k grows; neither happens. The first seam breaks whatever a reader was tracking, and the chunks after it are reading a text that was already broken. Assembly of independently valid chunks does not produce coherence at any scale, because the property that fails is binary at the first join, not something later joins can make marginally worse.

Stage	Count
LC/RC pairs mined	3,894
Both halves attested	131
Attested at three-word halves	27
Attested at four-word halves	0

Table 1: Material scarcity in mined LC/RC pairs, out of 272,000 attested English bigrams and 34,688 attested four-grams checked at the four-word-half stage. The count reaches zero before spans reach phrase length.

6 Why the ceiling is there

We mined candidate LC/RC pairs from attested English bigrams and four-grams; Table 1 shows how many survive at each span length, roughly up to where connected prose begins. This is an exhaustive count, not a sample with a confidence interval: usable material shrinks by roughly an order of magnitude at each step up in span length and hits zero before ordinary phrase length.

Figure 1b shows the same shortage from a different angle: every letter placed in a palindrome is placed twice, once from each direction, and both placements have to land inside a real word, so forward coverage times reverse coverage bounds what survives. Free letters also cost more bits under a dictionary model than under ordinary English (Shannon, 1951), though that framing carries an assumption the coverage number does not, so

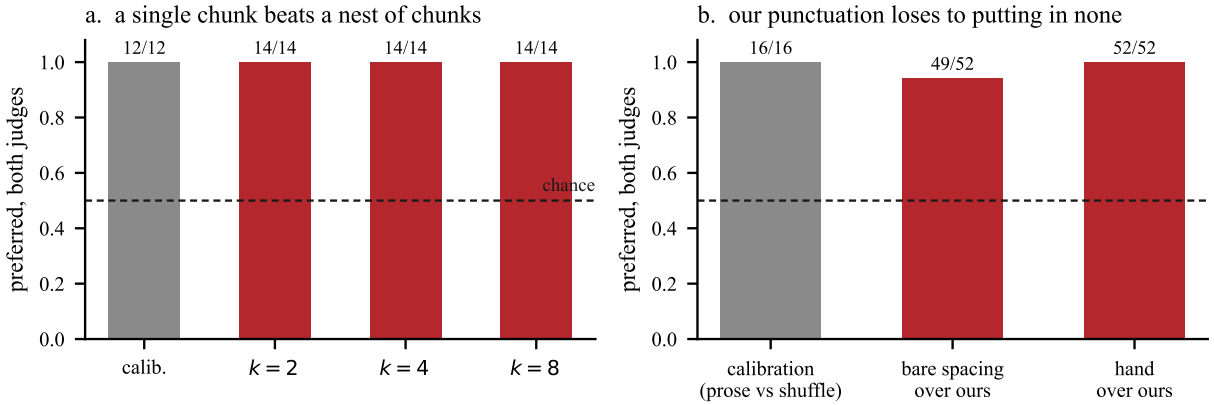

Figure 2: (a) Blind pairwise preference for a single chunk over a k-chunk nest built from the same material, at $k \in \{2, 4, 8\}$, alongside shuffle-versus-prose calibration. The single chunk wins every decision at every k; the collapse happens at the first seam and does not deepen. (b) Absolute and pairwise coherence under four punctuation treatments applied to the same 26 palindromes with identical letters in identical order: hand punctuation, post-hoc model marking, bare spacing, and our own search-time module, which loses to bare spacing.

we set it aside.

More compute does not help, in the direction that matters: Figure 1c shows throughput falling as vocabulary grows, with an exponent close to -1 – doubling the dictionary roughly halves the rate the search produces candidates, so a bigger dictionary or more search time will not close the gap in Table 1.

7 Punctuation is a confound

A palindrome search returns a run of words with no punctuation, and inserting marks before it reads as a sentence is usually invisible in reported results: two systems can return identical letters in identical order and still be judged on how a human or a model happened to punctuate the output.

We isolated this by taking the same 26 catalogued palindromes – identical letters, identical order – and varying only the marks: hand punctuation, post-hoc marking by a language model given the finished text, bare spacing with no marks at all, and our own search-time punctuation module. Absolute 0–3 coherence scores over the 26 texts were 2.31 for hand, 2.08 for post-hoc model marking, 1.31 for bare spacing, and 0.73 for our own module. Blind pairwise comparison, run under the same protocol as Section 5 with calibration at 16 of 16 (16/16), confirms the ordering: bare spacing beats our module 49 times out of 52 (49/52), and hand punctuation beats our module 52 out of 52 (52/52), with the two judges agreeing on every item. Left-pick rates of 27/52 and 26/52 rule out position bias.

Our punctuation module is worse than inserting none. The cause is a scoring artifact in the module itself: its segmentation objective adds a fixed reward per segment, so splitting text is free from the optimizer's point of view, and the dynamic program that chooses break points buys as many fragments as it can. "A man, a plan, a canal: Panama" comes out as "A, man a plan. A canal panama" – graded 3 by a human reader against a 0 for the module's own output on the same letters.

The fix is to stop deciding sentence boundaries inside the search. Hand the finished, unpunctuated word run to a language model with a single instruction: add marks, change no letter. This beats bare spacing and comes within 0.23 of hand punctuation on the absolute scale. Across 52 such requests to two different models, not one reply altered a letter of the underlying palindrome, so the constraint that the whole search exists to satisfy was never put at risk by handing punctuation to a separate model. Any judged comparison of palindrome output partly judges whoever chose the commas; only same-presenter comparisons are free of that confound.

8 Conclusion

Naming the overhang turns a constraint useless over prefixes into an ordinary finite-state one that existing decoding machinery can use directly. It does not remove the limits underneath. Progress per placement falls off for long one-sided units, composition breaks coherence at the first seam, and the ceiling moves the wrong way with vocabulary size. A punctuation module optimized on the wrong objective scores below no punctuation.

Limitations

The seam result in Section 5 rests on 21 palindrome items and 6 calibration items. Calibration establishes that the judging protocol has power, and both judges were unanimous, but the measurement is a ranking between two texts, not an absolute coherence score. A single chunk beating an eight-chunk nest 14 times out of 14 is consistent with both being unreadable, and inspection of the transcripts suggests they largely are; the result says the nest is worse, not that the single chunk is good.

The punctuation result in Section 7 covers 26 items, and the four variants of each item are not independent draws – they are the same letters marked four different ways, so the comparisons share all their underlying content. The hand-punctuated condition is ours, written to match how these palindromes are conventionally printed, with letters asserted identical to the other conditions rather than independently verified by a third party. A judge preferring human-punctuated text over a machine module is not a surprising outcome on its own, and some of the measured gap may reflect fluency of intent in how the marks were chosen rather than correctness of where they fall.

The vocabulary scaling exponent in Section 6 comes from 5 points over a 32-fold span run as a single job. It is a clean fit, but it is one job, not a replicated series, and we make no claim about how this exponent would behave outside the measured range.

The conservation measurements in Section 3 are taken on our own vocabulary and our own search implementation. We have not varied either independently, so we cannot say how much of the settled/net divergence is a property of palindrome construction in general versus a property of this particular candidate pool.

Automated judging needs its own gate. Of five local language models asked to separate real prose from its own word shuffle, three could not reliably do so, and mean-gap scoring is the weaker criterion for deciding whether a judge is usable: one model that passed a mean-gap threshold still reversed a result that blind pairwise comparison settled 52 out of 52 the other way, because it rated a large share of shuffled word salad as coherent while a stricter judge rated almost none. Every judged number in this paper comes from instruments checked against calibration items for this reason, and a reader applying a different judge should expect to need the same check.

An earlier version of this work reported a sharp yield cliff at a specific palindrome length. That claim has been retracted. It was a detection-floor artifact: the expected count in the bands read as "zero" was between 0.14 and 1.07 events under the null of no effect, so observing zero had a probability between 0.34 and 0.87 even if nothing changed at that length. We report the retraction here because the same failure mode – reading an artifact of small counts as a structural finding – is the risk this paper is otherwise trying to avoid.

References

Emmanuel Anaya Gonzalez, Sairam Vaidya, Kanghee Park, Ruyi Ji, Taylor Berg-Kirkpatrick, and Loris D'Antoni. 2025. Constrained sampling for language models should be easy: An MCMC perspective. *Preprint*, arXiv:2506.05754.

Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould. 2017. Guided open vocabulary image captioning with constrained beam search. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)*, pages 936–945.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. 2022. Efficient training of language models to fill in the middle. *Preprint*, arXiv:2207.14255.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. 2024. The reversal curse: LLMs trained on "a is b" fail to learn "b is a". In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Silvio Calderaro, Alessio Miaschi, and Felice Dell'Orletta. 2025. The OuLiBench benchmark: Formal constraints as a lens into LLM linguistic competence. In *Proceedings of the Eleventh Italian Conference on Computational Linguistics (CLiC-it 2025)*, pages 124–133.

Chris Donahue, Mina Lee, and Percy Liang. 2020. Enabling language models to fill in the blanks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pages 2492–2501.

Lukas Edman, Helmut Schmid, and Alexander Fraser. 2024. CUTE: Measuring LLMs' understanding of their tokens. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, pages 3017–3026.

Cristina Gârbacea and Qiaozhu Mei. 2025. Why is constrained neural language generation particularly challenging? *Transactions on Machine Learning Research*. First circulated as arXiv:2206.05395.

Olga Golovneva, Zeyuan Allen-Zhu, Jason Weston, and Sainbayar Sukhbaatar. 2024. Reverse training to nurse the reversal curse. *Preprint*, arXiv:2403.13799.

Chris Hokamp and Qun Liu. 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, pages 1535–1546.

Ouail Kitouni, Niklas Nolte, Diane Bouchacourt, Adina Williams, Mike Rabbat, and Mark Ibrahim. 2024. The factorization curse: Which tokens you predict underlie the reversal curse and more. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Alexander K. Lew, Tan Zhi-Xuan, Gabriel Grand, and Vikash K. Mansinghka. 2023. Sequential Monte Carlo steering of large language models using probabilistic programs. *Preprint*, arXiv:2306.03081.

João Loula, Benjamin LeBrun, Li Du, Ben Lipkin, Clemente Pasti, Gabriel Grand, Tianyu Liu, Yahya Emara, Marjorie Freedman, Jason Eisner, Ryan Cotterell, Vikash Mansinghka, Alexander K. Lew, Tim Vieira, and Timothy J. O'Donnell. 2025. Syntactic and semantic control of large language models via sequential Monte Carlo. *Preprint*, arXiv:2504.13139.

Ximing Lu, Sean Welleck, Peter West, Liwei Jiang, Jungo Kasai, Daniel Khashabi, Ronan Le Bras, Lianhui Qin, Youngjae Yu, Rowan Zellers, Noah A. Smith, and Yejin Choi. 2022. NeuroLogic A*esque decoding: Constrained text generation with lookahead heuristics. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2022)*, pages 780–799.

Ning Miao, Hao Zhou, Lili Mou, Rui Yan, and Lei Li. 2019. CGMH: Constrained sentence generation by Metropolis-Hastings sampling. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI 2019)*, pages 6834–6842.

Peter Norvig. 2002. World's longest palindrome? 21,012 words. With a companion write-up, *The Algorithm*, at https://norvig.com/pal-alg.html. Builds on Dan Hoey's 1984 program.

Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Matt Post and David Vilar. 2018. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2018)*, pages 1314–1324.

Lianhui Qin, Sean Welleck, Daniel Khashabi, and Yejin Choi. 2022. COLD decoding: Energy-based constrained text generation with Langevin dynamics. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.

Claude E. Shannon. 1951. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64.

Honghua Zhang, Meihua Dang, Nanyun Peng, and Guy Van den Broeck. 2023. Tractable control for autoregressive language generation. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*.